Einführung in die Rechnerarchitektur

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Überblick der nächsten Wochen

=

- **Grundlagen**
  - Beschreibungen über „0“ und „1“ (Boolesche Algebra)
  - Beschreibungen von Schaltungen
- **Rechnen**
  - Digitale Schaltungen für Addition, Subtraktion, Multiplikation
- **Speichern & Steuern**
  - Sequentielle Schaltungen
  - Speicherelemente
  - Endliche Automaten
- **Prozessor**
  - Aufbau
  - Optimierungen
- **Entwerfen**
  - Synthese von allgemeinen Schaltungen
  - Logikminimierung

Grundlagen: Beschreibung von Schaltungen

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

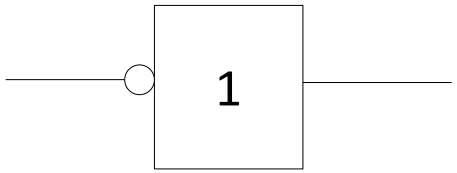
Follow @rbrtwll

DIN/IEC-Symbole #1

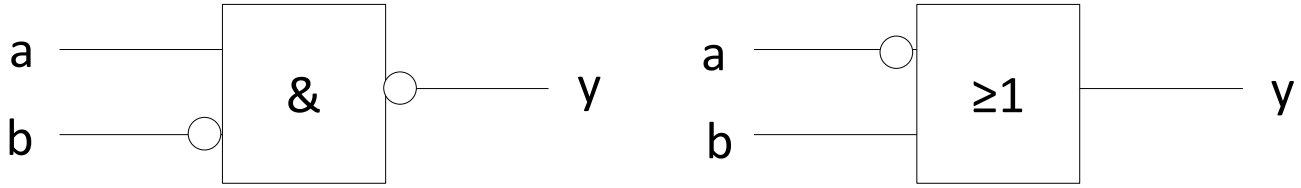
freie Symbole	Schaltsymbole nach DIN 40 700 Teil 14		amerikanische Symbole	logische Darstellung
	seit 1976	bis 1976		
UND	&			$x_1 \wedge \ldots \wedge x_n$
ODER	≥ 1			$x_1 \vee \ldots \vee x_n$
Anti-valenz	= 1			$x_1 \neq \ldots \neq x_n$
NAND	&			$\overline{x_1 \wedge x_2 \wedge \ldots \wedge x_n}$
NOR	≥ 1			$\overline{x_1 \vee x_2 \vee \ldots \vee x_n}$
Negation	1			$\overline{x_1}$

DIN/IEC-Symbole #2

- Negationssymbol darf auch am Eingang stehen



- Darstellungsmöglichkeiten der Implikation



EXOR-Realisierung

- EXOR nicht als Basisfunktion vorhanden

a	b	Exor
0	0	0
0	1	1
1	0	1
1	1	0

- Realisierung mit Hilfe von Und und Oder
- $y = \neg a * b \ + \ a * \neg b$
- Andere Realisierungen möglich (de Morgan)

Schaltkreis

- Hier: kombinatorische Schaltkreise
- Gerichteter, zyklenfreier Graph
- Knoten repräsentieren
  - Primäre Eingänge
  - Primäre Ausgänge
  - Gatter (i.d.R. basierend auf vorher festgelegter Gatterbibliothek)
- Kanten repräsentieren
  - Signale zwischen den Gattern bzw. primären Eingängen/Ausgängen
- Gängige Kostenmaße
  - Anzahl der Gatter (Größe)
  - Tiefe, d.h. Zahl der Gatter auf dem längsten Pfad von einem primären Eingang zu einem primären Ausgang (Geschwindigkeit)

Addition

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Addierer

<u>Gegeben</u>: 2 positive Binärzahlen $<a> = <a_{n-1} \ldots a_0>$ und $<b> = <b_{n-1} \ldots b_0>$,
Eingangsübertrag c aus $\{0,1\}$

<u>Gesucht</u>: Schaltkreis, der Binärdarstellung s von

$<a> + <b> + c$ berechnet

Wegen $<a> + <b> + c \leq 2\times(2^n - 1) + 1 = 2^{n+1} - 1$

genügen $n+1$ Ausgänge des Schaltkreises.

Der Halbaddierer (*HA*)

Der Halbaddierer dient zur Addition zweier 1-Bit-Zahlen *ohne* Eingangsübertrag.

Er berechnet die Funktion:

$ha : B^2 \rightarrow B^2$

mit $ha(a_0, b_0) = (s_1, s_0)$

mit $2s_1 + s_0 = a_0 + b_0$

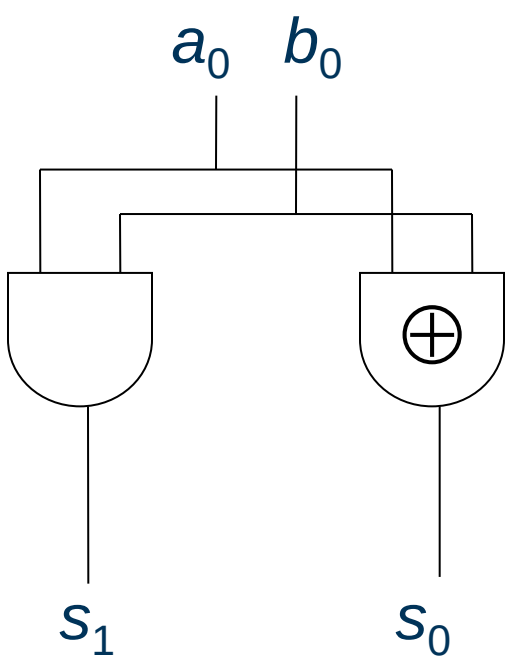
Funktionstabelle des HA

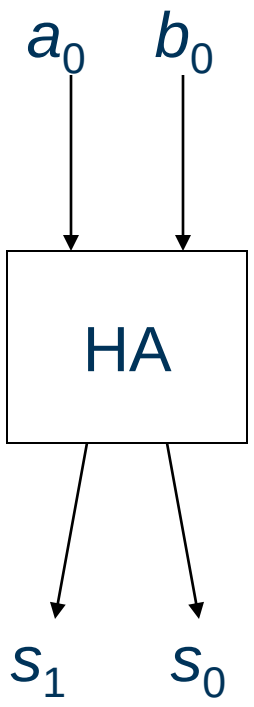
a_0	b_0	ha_1	ha_0
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Folglich:

$$ha_0 = a_0 \oplus b_0 \qquad ha_1 = a_0 \wedge b_0$$

Schaltkreis eines Halbaddierers





Kosten und Tiefe eines HA:

C(HA) = 2, depth(HA) = 1

Der Volladdierer (*FA*)

Der Volladdierer dient zur Addition zweier 1-Bit-Zahlen *mit* Eingangsübertrag.

Er berechnet die Funktion:

$fa: B^3 \rightarrow B^2$

mit $fa(a_0, b_0, c) = (s_1, s_0)$

mit $2s_1 + s_0 = a_0 + b_0 + c$

Funktionstabelle des FA

a_0	b_0	c	fa_1	fa_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Volladdierer als Funktion von HAs

Aus der Tabelle folgt:

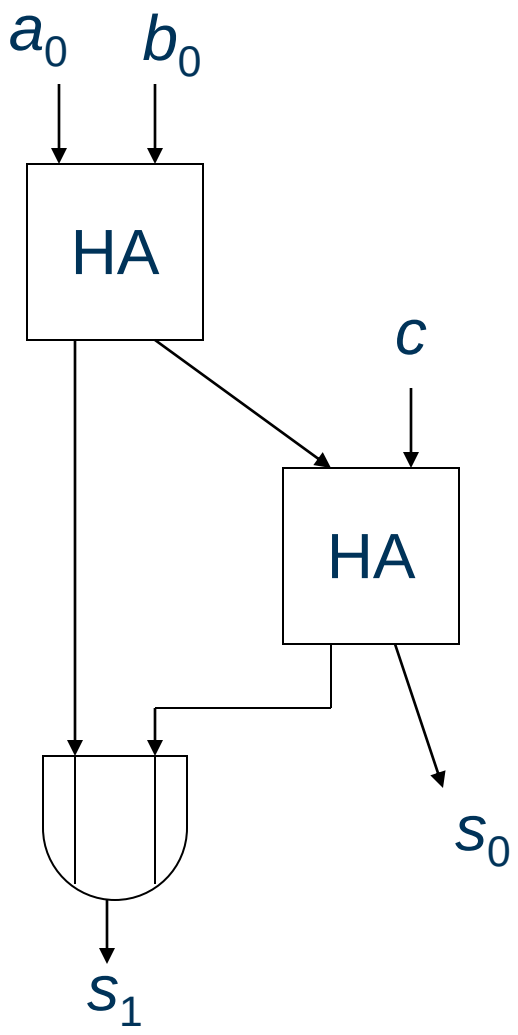
$$fa_0 = a_0 \oplus b_0 \oplus c = ha_0(c, ha_0(a_0, b_0))$$

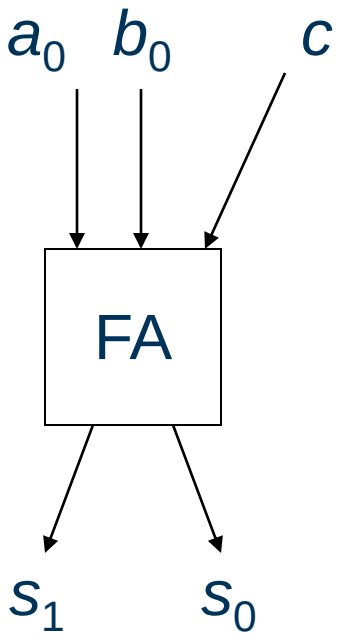
$$\begin{aligned} fa_1 &= a_0 \wedge b_0 \vee c \wedge (a_0 \oplus b_0) \\ &= ha_1(a_0, b_0) + ha_1(c, ha_0(a_0, b_0)) \end{aligned}$$

Kosten und Tiefe eines FA:

C(FA) = 5, depth(FA) = 3

Schaltkreis eines Volladdierers





Realisieren der Schulmethode: Carry Ripple Addierer (*CR*)

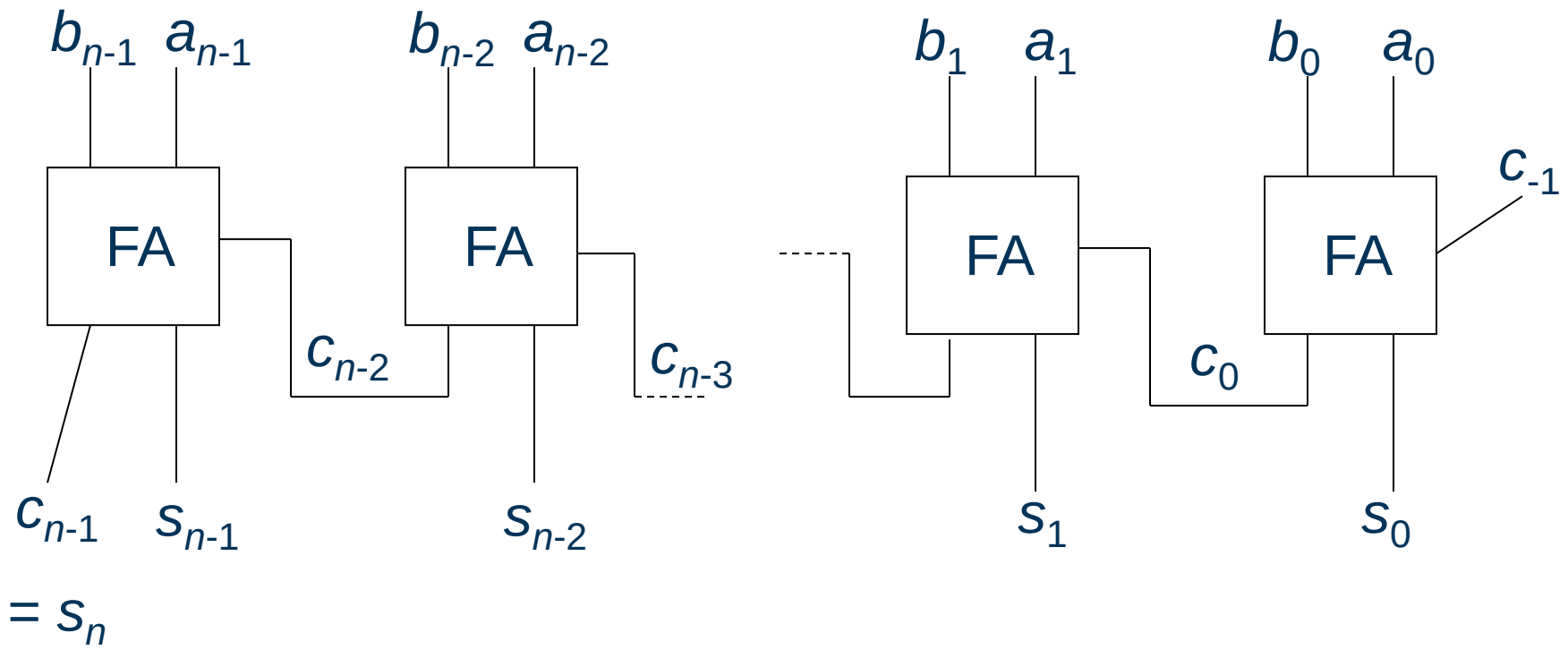
Hierarchisches Vorgehen:
(induktive Definition)

Für n=1: CR_1 = FA
Für n>1: Schaltkreis CR_n wie folgt definiert

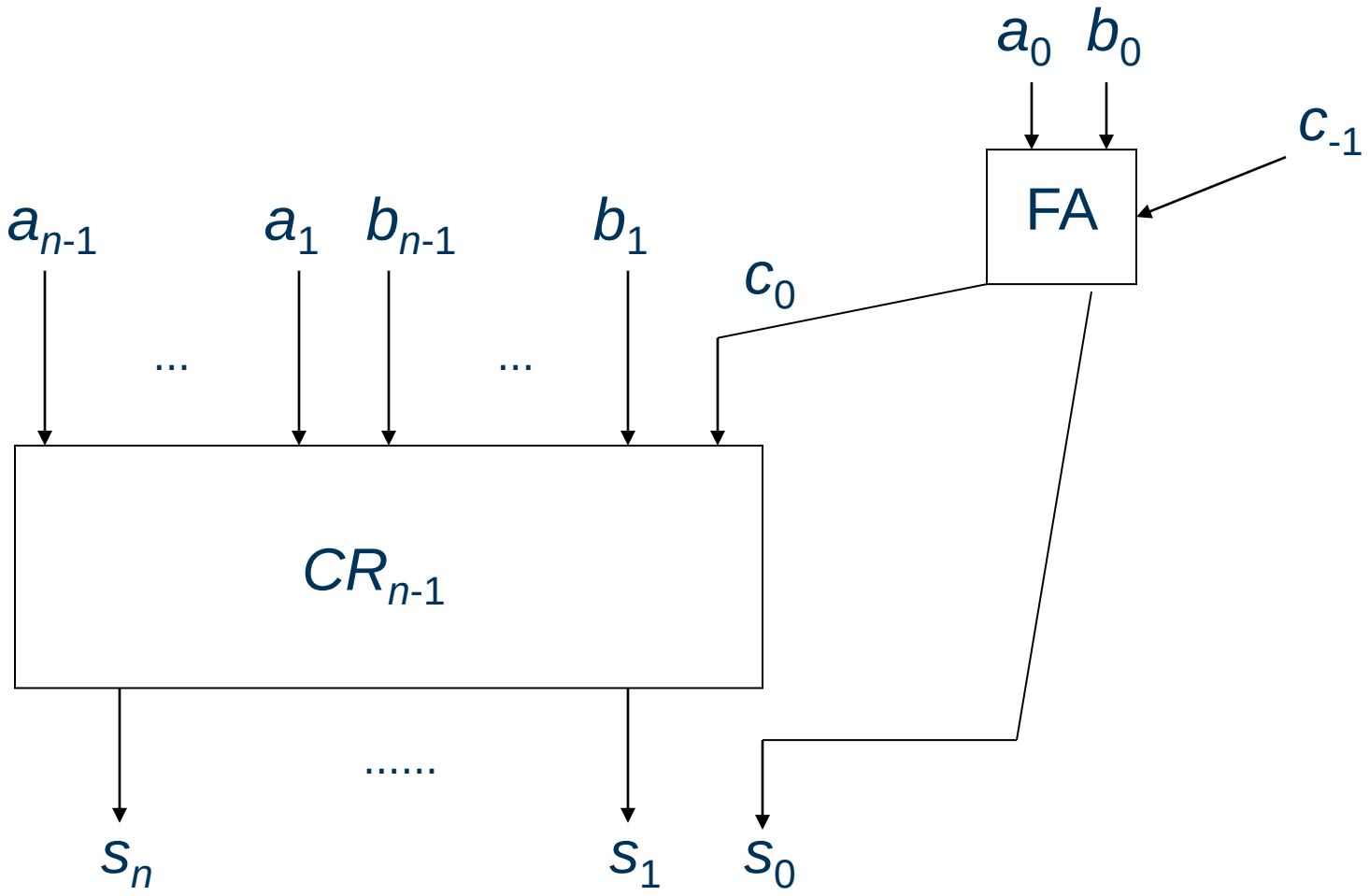
<u>Bezeichnung</u>:

Bezeichne den Eingangsübertrag mit c_{-1}, den Übertrag von Stelle i nach i+1 mit c_i.

Aufbau eines Carry Ripple Addierers



Schaltbild des n-Carry Ripple Addierers (CR_n)



Daten eines Carry Ripple Addierers

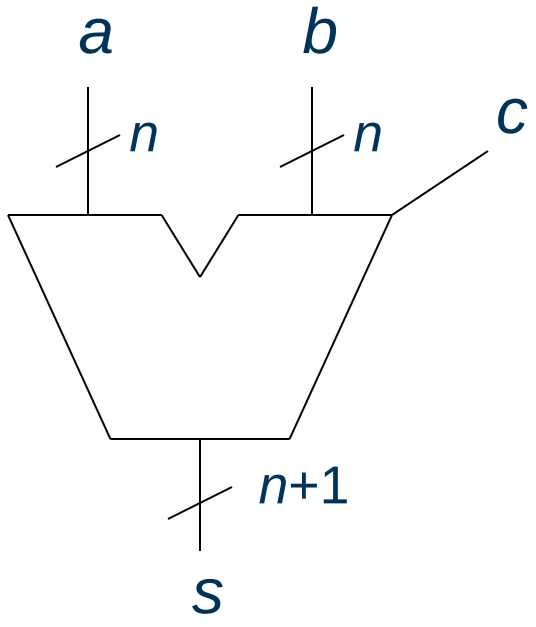
Kosten eines CR_n:

$C(CR_n) = n \cdot C(\text{FA}) = 5n$

Tiefe eines CR_n:

$\text{depth}(CR_n) = 3 + 2(n\text{-}1)$

Schaltbild eines n-Bit-Addierers:



Optimierte Addierer

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Conditional Sum Adder (CSA)

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

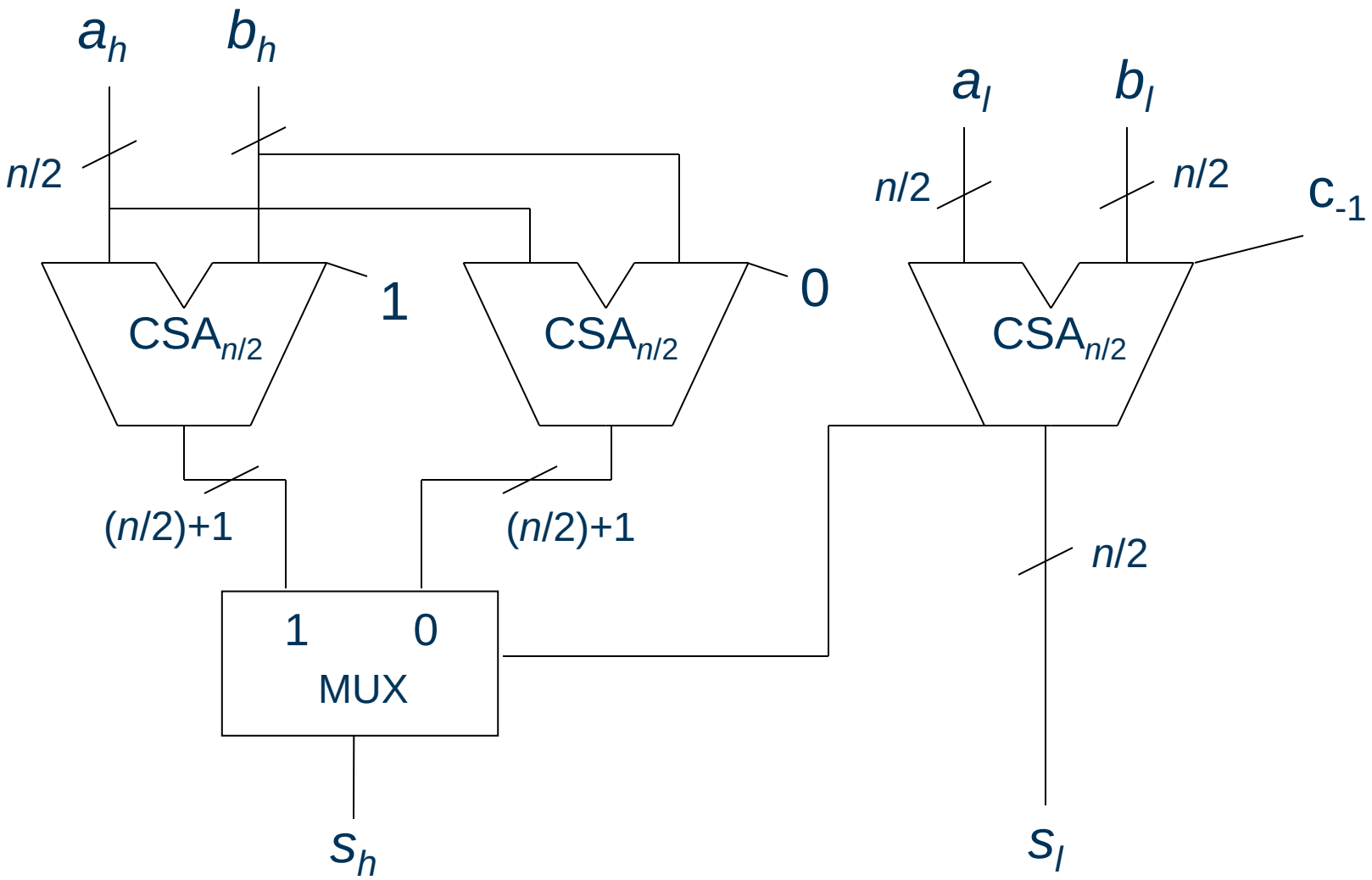
Follow @rbrtwll

Der Conditional Sum Adder (CSA)

<u>Idee</u>: Nutze Parallelverarbeitung, um Tiefe zu reduzieren!

$$
\begin{array}{rcl}
 & a_{n-1} \ldots a_{\frac{n}{2}} & a_{\frac{n}{2}-1} \ldots a_0 \\
+ & b_{n-1} \ldots b_{\frac{n}{2}} & b_{\frac{n}{2}-1} \ldots b_0 \\
+ & c_{\frac{n}{2}-1} & c_{-1} \\
\hline
 & s_n s_{n-1} \ldots s_{\frac{n}{2}} & s_{\frac{n}{2}-1} \ldots s_0
\end{array}
$$

Schaltbild des CSA_n



Subtraktion

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Behauptung

Zur Addition von $(n+1)$-Bit Zweierkomplementzahlen kann man $(n+1)$-Bit-Binäraddierer benutzen.

Der Test, ob das Ergebnis durch eine $(n+1)$-Bit-Zweierkomplementzahl darstellbar ist, d. h. ob das Ergebnis aus $R_n = \{-2^n, ..., 2^n-1\}$ ist,
lässt sich zurückführen auf den Test $c_n = c_{n-1}$.

Subtrahierer

Wegen $-[b]=[\overline{b}]+1$ kann $[a]-[b]$ zurückgeführt

werden auf $[a]+[\overline{b}]+1$.

→ Schaltkreis für Subtrahierer aus Addiererschaltkreis
→ kombinierter Addierer/Subtrahierer

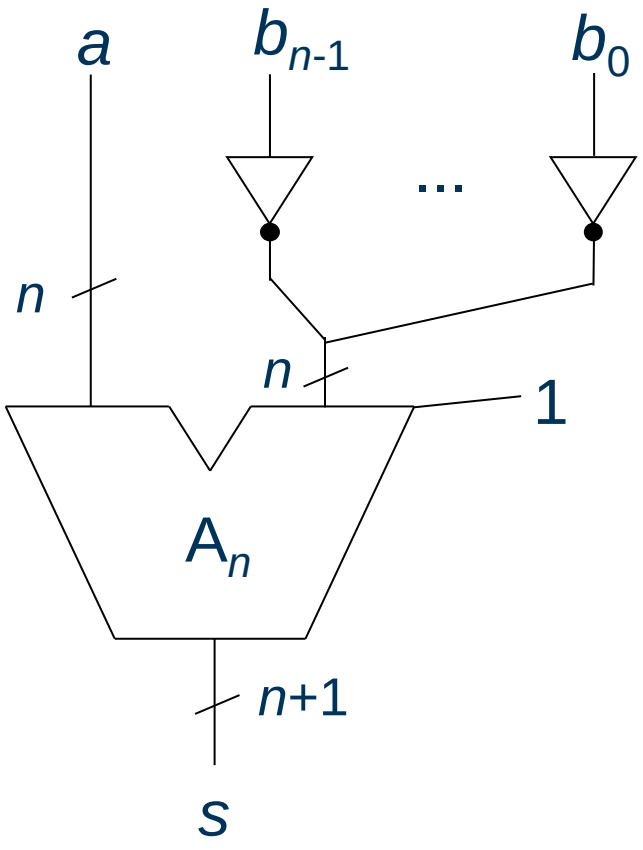
Beispiel

$$[a] = [0110] = 6_{10}, \qquad [b] = [0111] = 7_{10}, \qquad [\bar{b}] = [1000]$$

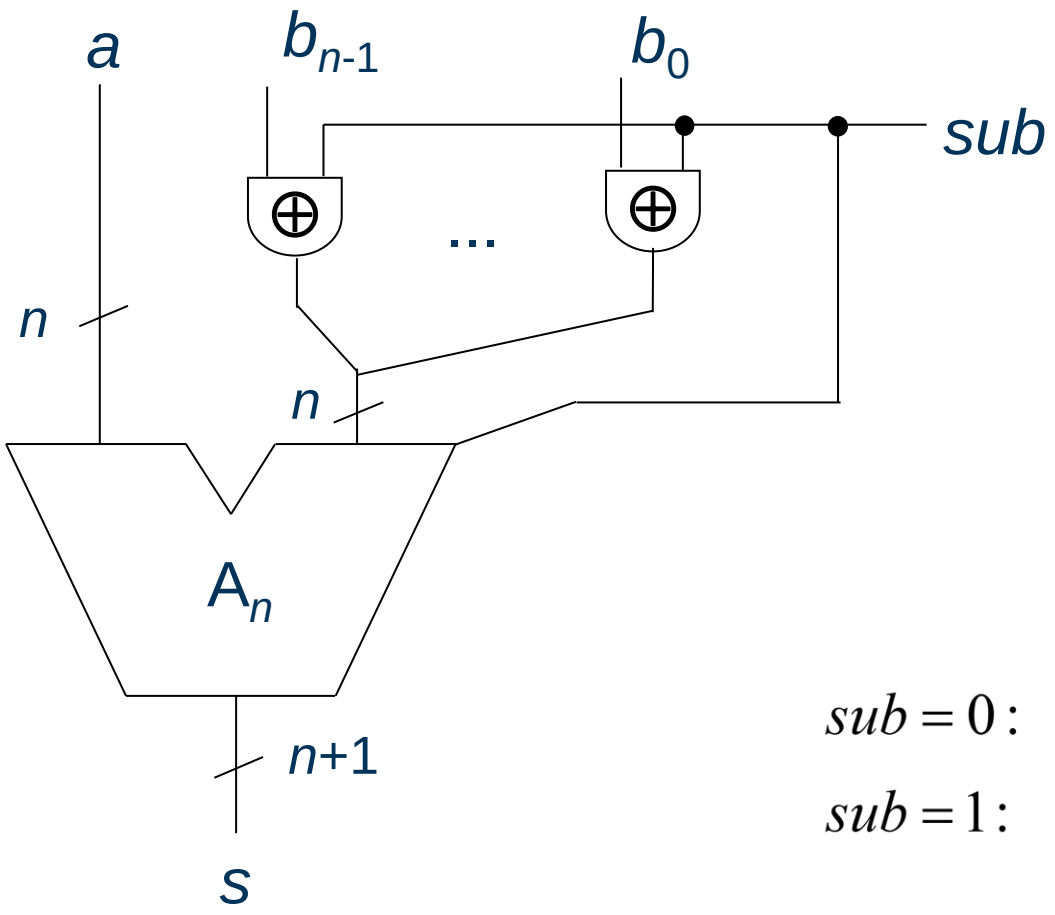
$$\begin{array}{cccc} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ & & & 1 \\ \hline 1 & 1 & 1 & 1 \end{array}$$

$$= (-1)_{10}$$

Schaltbild eines Subtrahierers



Schaltbild für einen Addierer/Subtrahierer



$$b_i \oplus 0 = b_i$$

$$b_i \oplus 1 = \overline{b_i}$$

$$sub = 0: \quad [a] + [b] + 0$$

$$sub = 1: \quad [a] + [\overline{b}] + 1 = [a] - [b]$$